

Installing ESP32-microros Components

Installing ESP32-microros Components

1. Downloading the microros Components
2. Activating the ESP-IDF Development Environment
3. Installing Dependencies
4. Modifying the microros Configuration
5. Compiling and Generating the microros Static Library
6. How to Use

Note: Since the `micro_ros` component is hosted on GitHub, downloading the files may fail due to network issues. The bundled virtual machine has already downloaded and compiled the microros components, so there is generally no need to reinstall the ESP32-microros components.

1. Downloading the microros Components

Download the microros components to `~/esp/Samples/extra_component`. Here, we take the humble version as an example.

```
mkdir -p ~/esp/Samples/extra_components
cd ~/esp/Samples/extra_components
git clone -b humble https://github.com/micro-ROS/micro_ros_espidf_component.git
cd micro_ros_espidf_component
```

`micro_ros` component source code URL: https://github.com/micro-ROS/micro_ros_espidf_component

2. Activating the ESP-IDF Development Environment

Run the following command in the terminal.

```
source ~/esp/esp-idf/export.sh
```

Note: Every time you open a new terminal, you must first activate the ESP-IDF development environment before you can compile ESP-IDF projects.

3. Installing Dependencies

Run the following command in the terminal.

```
pip3 install catkin_pkg lark-parser empy colcon-common-extensions
```

Note: Make sure you activate the ESP-IDF development environment before running the above command.

4. Modifying the microros Configuration

The default microros configuration only supports 1 node, 2 publishers, 2 subscribers, 1 service, 1 client, and 1 history.

If you find it insufficient, you can modify the microros configuration. If you do not modify it now and later find that it does not meet your requirements, you can run the following command to clear the files generated by the microros build, and then recompile to generate the microros static library.

```
cd
~/esp/Samples/extra_components/micro_ros_espidf_component/examples/int32_publisher
idf.py clean-microros
```

Modify the colcon.meta file in the micro_ros_espidf_component directory, find the "rmw_microxrcedds" section, and modify it according to your actual needs. For convenience, change the publishers, subscribers, and history to 3.

Note: Since the performance of the microcontroller is limited, just meeting the requirements is enough. Avoid wasting resources and affecting communication speed.

```
"rmw_microxrcedds": {
  "cmake-args": [
    "-DRMW_UXRCE_XML_BUFFER_LENGTH=400",
    "-DRMW_UXRCE_TRANSPORT=udp",
    "-DRMW_UXRCE_MAX_NODES=1",
    "-DRMW_UXRCE_MAX_PUBLISHERS=3",
    "-DRMW_UXRCE_MAX_SUBSCRIPTIONS=3",
    "-DRMW_UXRCE_MAX_SERVICES=1",
    "-DRMW_UXRCE_MAX_CLIENTS=1",
    "-DRMW_UXRCE_MAX_HISTORY=3"
  ]
},
```

5. Compiling and Generating the microros Static Library

Then run the following commands in the terminal.

```
cd
~/esp/Samples/extra_components/micro_ros_espidf_component/examples/int32_publisher
idf.py set-target esp32s3
```

After the compilation is complete, "Configuring done" and "Generating done" are displayed.

```
-- Configuring done
-- Generating done
-- Build files have been written to: /home/.../esp/Samples/extra_components/micro_ros_espidf_component/examples/int32_publisher/build
```

At this point, the system will automatically download the required files and compile to generate the libmicroros.a static library.

```
ls ../../*.a
```

```
~/esp/Samples/extra_components/micro_ros_espidf_component/examples/int32_publisher$ ls ../../*.a
../../libmicroros.a
```

6. How to Use

After the libmicroros.a static library is compiled and generated, the installation is complete. To use it, you only need to add extra_components to the project's external components.

Add the following content to the root CMakeLists.txt file of the project, replacing "../../extra_components" according to your actual path.

```
set (EXTRA_COMPONENT_DIRS "../../extra_components")
```

If you need to modify the microros compilation parameters, you can run the following command to clear the microros cache, and then recompile to generate the libmicroros.a static library.

```
idf.py clean-microros
```